

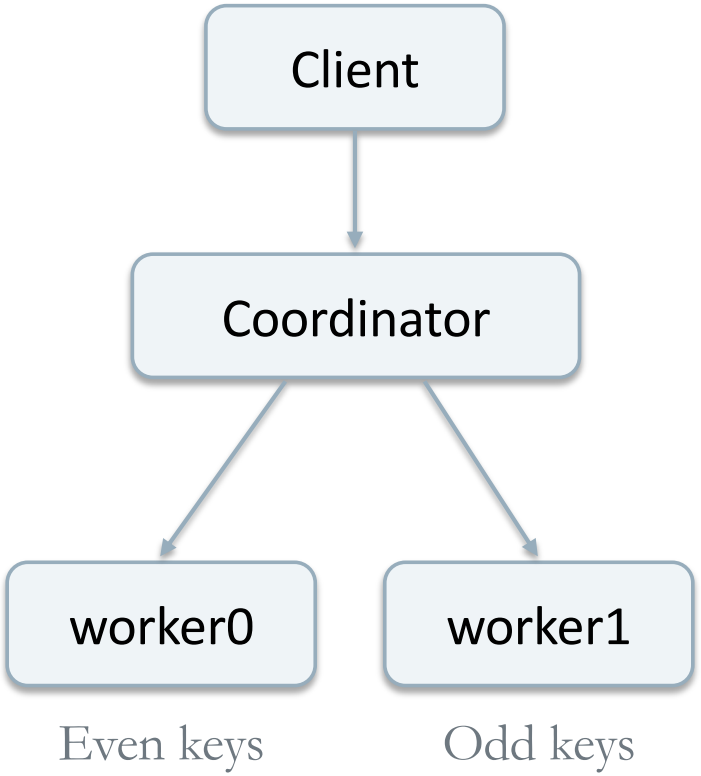
POLITECNICO
MILANO 1863

Actor Model – Akka
Evaluation Lab 2026

Luca Mottola
luca.mottola@polimi.it
https://mottola.faculty.polimi.it

Overview

- Implement **Distributed Counters**
- Four types of actors are involved
  - **Clients** submits records and requests reports
  - A **coordinator** routes records and combines replies
  - Two **workers** maintain a local count and sum
- All requests pass through the coordinator



Records and Workers

- Route **RecordMsg(key, value)** by key parity: even to worker0, odd to worker1
- Assume **non-negative keys and values**; every supplied record is valid
- Each worker maintains **count and sum**, both initially zero
  - Count is the number of (key, value) pairs since the last reset
  - Sum is the arithmetic sum of all current values
- **Repeated keys** count as new records; **zero values** increase count, but not sum

Reports and Reset

- **ReportMsg(reset)** asks the coordinator for one combined reply with **count and sum** across both workers
  - **reset = false:** return totals without changing them
  - **reset = true:** return **pre-reset** totals, then clear both workers before routing later records
- An empty window is **count = 0, sum = 0**
- When prompted for data of a report, the worker **emulates** a processing time **without blocking**

Collecting a Report

- Start a report: save the requester, become **collecting**, and query both workers
- While collecting, **buffer new records and reports**; continue handling worker replies
- Wait for **one result from each worker**, in either order
- Reply to the original requester, become **ready**, and **unbuffered earlier messages**
  - Any ordering of processing for buffered messages is admissible

Resetting: An Example

Coordinator step	State
Before report	worker0: (2, 15); worker1: (1, 7)
Start Report(true)	Both workers capture their counters and reset
Collecting	Record(0, 3), then Report(false), are buffered
First report completes	Reply: count = 3, sum = 22
Release record	worker0 becomes count = 1, sum = 3
At the next report	Reply: count = 1, sum = 3

Order shown is the coordinator's processing order.

Requirements and Assumptions

- Client use the ask pattern for reports
- **No blocking in actors**
- Use the coordinator as sender for **all commands to workers**
- Do **not duplicate** worker state at the coordinator
- Assume valid input, reliable delivery, and actors that remain running

Code

- Provided: immutable messages, actor templates, two client test scripts, and main method
- Includes a setup with two clients and pre-defined test scripts
  - Do not change startup code unless you know what you are doing…
- Keep the package name, public APIs, and supplied client code unchanged
  - You can add messages, classes, …

Rules and Guidelines

- Proper use of **actor abstractions** is the key grading criterion
- Complete **README.md** with
  - Your group identifier and the name of each member
  - **At most 200 words** describing who sends which messages, to whom, when, and how
    - Do not just repeat the assignment!
- Submit the entire source project as **akka-groupXX.zip**
  - XX is the group identifier from the registration document
  - Submit through the **member at the registered contact email**
- Exclude **target/** output and dependency JARs